

A lightweight implementation of dependently sized types for a functional hardware description language

J. Sérot¹

Institut Pascal, Université Blaise Pascal/CNRS, France, jocelyn.serot@univ-bpclermont.fr

Abstract. We describe the type system integrated to the last version CAPH programming language, a domain specific programming language dedicated to the implementation of dataflow algorithms on FPGA. This type system supports the notion of *sized* integers and a limited form of type dependency. These features significantly augment the expressive power of the language and its ability to accurately and efficiently describe computations to be implemented in hardware.

1 Context and Motivations

CAPH [1–3] is a domain-specific language for describing and implementing stream-processing applications on reconfigurable hardware, such as FPGAs. The CAPH compiler generates target-independant, ready-to-synthesize VHDL code from high-level descriptions of signal or image processing applications. CAPH relies upon the actor/dataflow model of computation. Applications are described as networks of purely dataflow actors exchanging tokens through unidirectional channels.

The CAPH language is actually built from two layers : an actor description language (ADL) for describing the *behavior* of dataflow actors and a *network description language* (NDL) for describing the *structure* of dataflow networks¹. The ADL describes the behavior of individual actors as a set of *transition rules* involving pattern matching on input values and local variables. The NDL is a small, purely functional, polymorphic and higher-order language in which graphs are described by applying actors, interpreted as functions, to values representing wires.

A very simple CAPH program and its corresponding dataflow graph representation is given in Fig. 1². For each input token carrying an integer v , it produces an output token carrying the integer $v \times 2 + v \times 3$. Lines 1-14 contains the description of the three involved actors. The **dup** actor duplicates its input, the **scale** actor multiplies its input by a given factor and the **add** actor adds its two inputs. Lines 16 and 17 define the network I/Os. The network itself is described in lines 19-20. Each **net** declaration introduces a new wire (or pair of wires in line 19) and binds a name to it for subsequent reuse³. The description is complete when all outputs declared in the IO section have been bounded. Note that the **scale** actor has been defined as accepting a *parameter* (lines 11) and that the corresponding parameter value is specified when instanciating the actor at the network level (lines 20). The **dup** and **add** actors have no parameter.

In the sequel, we will focus on the typing properties of the network description language. For this, we will essentially view actors as “black boxes”, only described by the type of their inputs, outputs and parameters. The NDL can then be described with the grammar and type system of Fig 2⁴. From this perspective, it is essentially a call-by-value λ -calculus, augmented with pairs, **let** bindings, builtin types **int** and **bool** and

¹ There’s in fact a extra layer, for describing global constants and functions but this feature is essentially orthogonal to the issues discussed here and will be ignored in the sequel.

² The graph on the right has actually been produced automatically from the source code on the left by the CAPH compiler.

³ It is also possible to define wiring functions in CAPH, *i.e.* functions taking actors or other functions and wires and applying the formers to the latters. This will not be described here.

⁴ This is actually a simplified versions of the CAPH NDL, in which features not relevant to the issues discussed here have been omitted.

```

1  actor dup in (i:$t) out (o1:$t, o2:$t)
2  rules
3  | i:x -> (o1:x, o2:x)
4  ;
5
6  actor add in (i1:int, i2:int) out (o:int)
7  rules
8  | (i1:x, i2:y) -> o:x+y
9  ;
10
11 actor scale (k: int) in (i:int) out (o:int)
12 rules
13 | i:x -> o:x*k
14 ;
15
16 stream inp: int from "inp.txt";
17 stream outp: int to "res.txt";
18
19 net (x,y) = dup inp;
20 net outp = add (scale 2 x, scale 3 y);

```

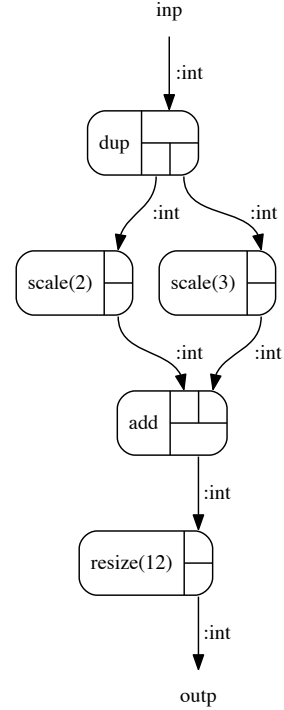


Fig. 1. A simple CAPH program

for which actors are primitive functions. Within this simplified framework, typing is completely standard. The only noteworthy point is the distinction, in the type signature, between actors accepting parameters and actors without parameters. The latter are assigned a type like $\tau'_1 \times \dots \times \tau'_m \rightarrow \tau''_1 \times \dots \times \tau''_n$, where $\tau'_1, \dots, \tau'_m$ are the types of the inputs and $\tau''_1, \dots, \tau''_n$ the types of the outputs. The former will be assigned a type like $\tau_1 \times \dots \times \tau_k \rightarrow \tau'_1 \times \dots \times \tau'_m \rightarrow \tau''_1 \times \dots \times \tau''_n$, where $\tau_1, \dots, \tau_p$ are the types of the parameters. For example, the three actors involved in the program of Fig. 1 respectively have type

- **dup** : $\forall \alpha. \alpha \rightarrow \alpha \times \alpha$ ⁵
- **scale** : $\text{int} \rightarrow \text{int} \rightarrow \text{int}$
- **add** : $\text{int} \times \text{int} \rightarrow \text{int}$

It is then straightforward to compute the types of any wire using Hindley-Millner type inference (as shown on the right side of Fig. 1) and thereby check the coherence of the network.

In a context where programs have to be implemented in hardware, the simplified version of the NDL introduced above lacks a crucial feature : the ability to specify the *size* (in bits) of the manipulated values (integers here). Contrary to software implementations, in which the size of integers can only take a small set of values (8, 16, 32 and 64 typically), hardware implementations can choose to implement integers of *any* size. In fact, selecting an “optimal” size, minimizing resource consumption⁶ while keeping an acceptable precision,

⁵ In the program of Fig. 1, type variables are denoted with a "\$" prefix.

⁶ Every bit will effectively allocate a flip-flop, related logic and will require additional routing.

```

program ::=      program acts ios nets
act ::=          actor id [params] ains aouts ...
param, ain, aout ::= id : tyexpr
io ::=           stream id tyexpr (from|to) string
net ::=          net ⟨binding1, ..., bindingn⟩
binding ::=      pat = expr
expr ::=         | var
                 | const
                 | ( expr1, ..., exprn )
                 | expr1 expr2
                 | function pat → expr
                 | let ⟨binding1, ..., bindingn⟩ in expr
pat ::=          | var
                 | (pat1, ..., patn)
tyexpr ::=       int | bool
                 | tvar
                 | tyexpr1 × ... × tyexprn
                 | tyexpr1 → tyexpr2

```

Fig. 2. The CAPH network description language (simplified)

is a key issue in digital design. Since this information cannot, in the general case, be inferred by the compiler, it must be provided by the programmer. Reflecting it in the type system at the source language level is the more effective and safe way to do this.

In our case, what is needed is a way to *parameterize* the `int` type with a size annotation, so that we can, for instance, declare and manipulate values with type `Int⟨8⟩` or `Int⟨16⟩`⁷. This, in turn, immediately calls for a need to define actors able to operate on integers with a variable size. It is not realistic, for example, to have to define separate (and obviously redundant) versions of the `add` actor operating on `Int⟨8⟩`, `Int⟨16⟩`, `Int⟨32⟩`, *etc.* In other words, our language should allow the definition of actors having, typically, the following type signature :

- `add` : $\forall n. \text{int}\langle n \rangle \times \text{int}\langle n \rangle \rightarrow \text{int}\langle n \rangle$
- `scale` : $\forall n. \text{int}\langle n \rangle \rightarrow \text{int}\langle n \rangle \rightarrow \text{int}\langle n \rangle$

But the notion of *size genericity* introduced above is sometimes not enough. Suppose, for instance, that we modify the program in Fig. 1 to use size-generic actors, as illustrated in Fig. 3. Suppose that the target application is intended to run in a hardware context which imposes that the network input and output (`inp` and `outp`) have type `Int⟨16⟩` and `Int⟨8⟩` respectively⁸. Then the program in Fig. 3 does not type-check (expecting `outp` having type `Int⟨8⟩` whereas its inferred type is `Int⟨16⟩`). A solution would be to modify the `add` actor so that its output is forced to have type `Int⟨8⟩`. But this breaks both the genericity of the actor and the more general principle of modularity. Another solution is to insert, between the output of the `add` actor and the network output `outp`, an actor `resize`, whose function is precisely to adjust the size of its argument. A possible signature for such an actor would be :

`resize8` : $\forall n. \text{int}\langle n \rangle \rightarrow \text{int}\langle 8 \rangle$

⁷ In fact, the problem is a little bit more complex since hardware description languages, such as VHDL, also make a distinction between *signed* and *unsigned* integers. For simplicity, and because this issue is largely orthogonal to the problem discussed here – it can be solved in the context of standard parametric polymorphism using a classical “trick” known as phantom types –, we will ignore it in this presentation.

⁸ There’s no such thing as variable size integers at the hardware level...

```

1 actor dup in (i:$t) out (o1:$t, o2:$t) ...
2 actor add in (i1:int<s>, i2:int<s>) out (o:int<s>) ...
3 actor scale (k: int<s>) in (i:int<s>) out (o:int<s>) ...
4
5 stream inp: int<16> from "inp.txt";
6 stream outp: int<8> to "res.txt";
7
8 net (x,y) = dup inp;
9 net outp = add (scale 2 x, scale 3 y);

```

Fig. 3. A modified version of the program appearing in Fig. 1

But, of course, the need then quickly arises for a *generic* version of this actor, taking the resizing factor as a parameter, so that, in our case, the last line of the program in Fig. 3 could be written :

```
net outp = resize 8 (add (scale 2 x, scale 3 y));
```

For this, the type of the **resize** actor has to be something like

$$\forall n. k : \text{Int}\langle 8 \rangle \rightarrow \text{Int}\langle n \rangle \rightarrow \text{Int}\langle k \rangle$$

where the type of result depends on the value of the first argument (explicitely named k here), *i.e.* a *dependent type*.

In this paper, we describe a type system supporting i) the notion of sized types and size-generic functions ii) a limited form of type dependency, in which the sizes attached to some function types may depend on the value of some of its arguments. We show that both notions can be implemented in a very lightweight manner, without the complexities of more general approaches.

2 A dependently sized language

For simplicity, and because application to the CAPH network description language is straightforward, our description will be carried out using a simple, strict, higher-order and polymorphic functional language, $\mathcal{L}_{DS}$.

This language is described in Fig. 4. It is essentially a sugared version of a call-by-value lambda-calculus, augmented with pairs, **let** bindings, size annotations and a type refinement operator. In this presentation, for simplicity, and without loss of generality, there are only two type constructors, **bool** and **int**, and size annotations only apply to **int**.

There are three kinds of size expressions : constants, variables and references.

Size constants are used to define fixed-size types (such as **int**<16>) and size-monomorphic functions.

Size variables play the rôle of type variables in a classical HM-typed language; they appear whenever the size of the related constructor is still unknown and are used to define size-polymorphic functions. For this, the notion of type scheme is extended to accomodate, along with the list of generic type variables, a list of generic size variables. As a result, the general structure of type schemes is

$$\sigma ::= \forall \langle \alpha, \beta \rangle. \tau,$$

where α is a set of type variables and β is a set of size variables. When α (resp. β) is empty and there's no risk of ambiguity, we will simply write $\forall \beta. \tau$ (resp. $\forall \alpha. \tau$).

Some examples :

	Expression		Types
$e ::= x$	<i>constant</i>	$\tau ::= \text{Bool}$	
n		$\text{Int}\langle\zeta\rangle$	
(e_1, e_2)		$\tau_1 \times \tau_2$	
$e_1 e_2$		$\tau_1 \rightarrow \tau_2$	
$\text{fun } p \rightarrow e$		α	<i>type variable</i>
$\text{let } x_1 = e_1 \text{ in } e_2$		$\zeta ::=$	type size
$e \triangleright t$	<i>type refinement</i>	n	<i>size constant</i>
$p ::= x$	Pattern	β	<i>size variable</i>
$x : t$		$\bar{k}$	<i>size reference</i>
$t ::=$	Type expression		
$\text{bool} \mid \text{int}\langle s \rangle$		$k ::= 1 \mid 2 \mid \dots$	index
$t_1 \times t_2$			
$t_1 \rightarrow t_2$		$\sigma ::= \forall \langle \alpha, \beta \rangle. \tau$	type scheme
$s ::=$	Size expression		
n	<i>size constant</i>	$\Gamma ::= \emptyset \mid \{x:\sigma\}. \Gamma$	typing environment
x	<i>size variable</i>	$\Phi ::= \emptyset \mid \{x \mapsto \beta\}. \Phi$	variable environment
$@x$	<i>size reference</i>	$\Upsilon ::= \emptyset \mid \Upsilon.\{x\}$	<i>index environment</i>

Fig. 4. The $\mathcal{L}_{DS}$ language

- 0 has type $\text{Int}\langle\beta_1\rangle$, where β_1 is a (fresh) size variable,
- $\text{fun } x \rightarrow x+1$ has type $\forall\beta. \text{Int}\langle\beta\rangle \rightarrow \text{Int}\langle\beta\rangle^9$.
- $\text{fun } x \rightarrow \text{fun } y \rightarrow x, y+1$ has type $\forall\langle\alpha, \beta\rangle. \alpha \times \text{Int}\langle\beta\rangle \rightarrow \alpha \times \text{Int}\langle\beta\rangle$
- $\text{fun } (x:\text{int}\langle n \rangle) \rightarrow \text{fun } (y:\text{int}\langle n \rangle) \rightarrow x, y$ has type $\forall\beta. \text{Int}\langle\beta\rangle \rightarrow \text{Int}\langle\beta\rangle \rightarrow \text{Int}\langle\beta\rangle \times \text{Int}\langle\beta\rangle$

Size references can only appear in function types. In this context, they allow a size annotation, attached to the type of an argument or result of this function, to refer to the value of some previous argument. At the source level, this is done by using the name of the referred argument¹⁰. At the type level, explicit naming is replaced by a De Bruijn-like indexing mechanism, thus clearly distinguishing size variables and size references.

For example, the function corresponding to the **resize** actor introduced at the end of Sec. 1 can be written as¹¹ :

```
let resize = fun (k:int<m>) (x:int<n>) -> (x::int<@k>)
```

and will have type

$$\forall m, n. \text{Int}\langle m \rangle \rightarrow \text{Int}\langle n \rangle \rightarrow \text{Int}\langle \bar{1} \rangle$$

so that

```
resize 8 (0:int<16>)
```

has type

$$\text{Int}\langle 8 \rangle$$

⁹ Assuming the builtin operator $+$ has type $\forall\beta. \text{Int}\langle\beta\rangle \times \text{Int}\langle\beta\rangle \rightarrow \text{Int}\langle\beta\rangle$.

¹⁰ The " $@$ " prefix in the abstract syntax is to avoid confusion with size variables.

¹¹ Using, here and in the sequel, the syntax $\text{fun } p_1 \dots p_n \rightarrow e$ as a shorthand for $\text{fun } p_1 \rightarrow \dots \text{fun } p_n \rightarrow e$, and denoting $:$ the size refinement operator $\triangleright$.

Similarly, the function

```
fun (k1:int<m>) (k2:int<m>) (x:int<n1>,x:int<n2>) -> ((x:int<@k1>),(y:int<@k2>))
```

will have type

$$\forall m, n, p. \text{Int}\langle m \rangle \rightarrow \text{Int}\langle m \rangle \rightarrow \text{Int}\langle n \rangle \times \text{Int}\langle p \rangle \rightarrow \text{Int}\langle \bar{1} \rangle \times \text{Int}\langle \bar{2} \rangle$$

And a the function `fun (s:bool) (x:int<@s>) -> x` is ill-typed.

2.1 Typing

Typing rules for the language $\mathcal{L}_{DS}$ are given in Figs. 5 and 6.

Rules in Fig. 5 deal with **type expressions**. They are of the form $\Gamma, \Phi, \mathcal{Y} \vdash t \Rightarrow \tau, \Phi'$, which reads : in a context made of a *type environment* Γ , a *variable environment* Φ and an *index environment* $\mathcal{Y}$, typing the type expression t gives a type τ and an updated variable environment Φ' . The type and variable environment Γ and Φ are classical. The former contains bindings of variables, constants and builtin functions to type schemes. The latter is used for recording the variables occurring in size expressions for subsequent binding. Classically, we note $\emptyset$ the empty environment, $E.x : b$ the environment obtained by adding the binding $x \mapsto b$ to environment E and $E(x) = b$ or $x : b \in E$ if the binding $x \mapsto b$ appears in E . We will assume that there can be at most one binding having the same name in every environment. The index environment $\mathcal{Y}$ is just an ordered list of identifiers and is used to convert named references to DeBruijn-like indices. We here write $\mathcal{Y}.x$ for the environment obtained by simply appending identifier x to $\mathcal{Y}$ and $\{x_1, x_2, \dots, x_n\}$ the environment $\emptyset.x_1.x_2.\dots.x_n$. We write $\mathcal{Y}(x_i) = i$ if $\mathcal{Y} = \{x_1, x_2, \dots, x_n\}$ and $1 \leq i \leq n$ (*i.e.* if x appears, at position i in $\mathcal{Y}$) and $x \notin \mathcal{Y}$ otherwise (*i.e.* if x does not appear in $\mathcal{Y}$).

$$\begin{array}{c}
\frac{}{\Gamma, \Phi, \mathcal{Y} \vdash \text{bool} \Rightarrow \text{Bool}, \Phi} \text{R1} \quad \frac{}{\Gamma, \Phi, \mathcal{Y} \vdash \text{int}\langle n \rangle \Rightarrow \text{Int}\langle n \rangle, \Phi} \text{R2} \quad \frac{x:\beta \in \Phi}{\Gamma, \Phi, \mathcal{Y} \vdash \text{int}\langle x \rangle \Rightarrow \text{Int}\langle \beta \rangle, \Phi} \text{R3} \\
\\
\frac{x \notin \Phi \quad \text{fresh}(\beta)}{\Gamma, \Phi, \mathcal{Y} \vdash \text{int}\langle x \rangle \Rightarrow \text{Int}\langle \beta \rangle, \Phi.\{x:\beta\}} \text{R4} \quad \frac{\text{fresh}(\beta)}{\Gamma, \Phi, \mathcal{Y} \vdash \text{int} \Rightarrow \text{Int}\langle \beta \rangle, \Phi} \text{R5} \\
\\
\frac{x:\text{Int}\langle \cdot \rangle \in \Gamma \quad \mathcal{Y}(x) = k}{\Gamma, \Phi, \mathcal{Y} \vdash \text{int}\langle @x \rangle \Rightarrow \text{Int}\langle \bar{k} \rangle, \Phi} \text{R6} \quad \frac{\Gamma, \Phi, \mathcal{Y} \vdash t_1 \Rightarrow \tau_1, \Phi_1 \quad \Gamma, \Phi_1, \mathcal{Y} \vdash t_2 \Rightarrow \tau_2, \Phi_2}{\Gamma, \Phi, \mathcal{Y} \vdash t_1 \times t_2 \Rightarrow \tau_1 \times \tau_2, \Phi_2} \text{R7} \\
\\
\frac{\Gamma, \Phi, \mathcal{Y} \vdash t_1 \Rightarrow \tau_1, \Phi_1 \quad \Gamma, \Phi_1, \mathcal{Y} \vdash t_2 \Rightarrow \tau_2, \Phi_2}{\Gamma, \Phi, \mathcal{Y} \vdash t_1 \rightarrow t_2 \Rightarrow \tau_1 \rightarrow \tau_2, \Phi_2} \text{R7}
\end{array}$$

Fig. 5. Typing rules for $\mathcal{L}_{DS}$ (type expressions)

Rule R1 deals with the un-annotated type constructor `bool`.

Rules R2-R5 deal with the `int` constructor. In rules R3 and R4, the variable environment Φ keeps track of variable occurring in size expressions¹². Rule R5 deals with the special case in which the size annotation is omitted. We note $\text{Int}\langle \cdot \rangle$ the type $\text{Int}\langle \zeta \rangle$ when the size annotation ζ does not matter.

¹² Just as it does for type variables in a language supporting polymorphic type constructors.

Size references are dealt with in rule R6. This rule converts a *named* size reference to a De Bruijn-like index, using the index environment $\mathcal{I}$. The rule also checks that the type of the pointed argument is Int .

Rules R6 and R7, dealing with pair and function types are classical.

Rules in Fig 6 deal with **expressions**. Here the judgement $\Gamma, \Phi, \mathcal{I} \vdash e : \tau$ means that under the assumptions recorded in Γ, Φ and $\mathcal{I}$, expression e has type τ .

$$\begin{array}{c}
\frac{x:\sigma \in \Gamma}{\Gamma, \Phi, \mathcal{I} \vdash x : \text{Inst}(\sigma)} \text{TVAR} \qquad \frac{}{\Gamma, \Phi, \mathcal{I} \vdash \text{b} : \text{Bool}} \text{TBOOL} \qquad \frac{\text{fresh}(\beta)}{\Gamma, \Phi, \mathcal{I} \vdash \text{n} : \text{Int}(\beta)} \text{TINT} \\
\\
\frac{\Gamma, \Phi, \mathcal{I} \vdash e_1 : \tau_1 \quad \Gamma, \Phi, \mathcal{I} \vdash e_2 : \tau_2}{\Gamma, \Phi, \mathcal{I} \vdash (e_1, e_2) : \tau_1 \times \tau_2} \text{TPAIR} \qquad \frac{\Gamma, \Phi, \mathcal{I} \vdash e_1 : \tau \quad \Gamma.\{x:\text{Gen}(\tau, \Gamma)\} \vdash e_2 : \tau'}{\Gamma, \Phi, \mathcal{I} \vdash \text{let } x = e_1 \text{ in } e_2 : \tau'} \text{TLET} \\
\\
\frac{\Gamma, \Phi, \mathcal{I} \vdash e : \tau \quad \Gamma, \Phi, \mathcal{I} \vdash t \Rightarrow \tau', \Phi' \quad \tau <: \tau'}{\Gamma, \Phi, \mathcal{I} \vdash e \triangleright t : \tau'} \text{TCAST} \\
\\
\frac{\Gamma, \Phi, \mathcal{I} \vdash t \Rightarrow \tau_1, \Phi' \quad \Gamma.\{x:\forall. \tau_1\}, \Phi', \mathcal{I}.x \vdash e : \tau_2}{\Gamma, \Phi, \mathcal{I} \vdash \text{fun } (x : t) \rightarrow e : \tau_1 \rightarrow \tau_2} \text{TFUN} \\
\\
\frac{\Gamma, \Phi, \mathcal{I} \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \text{refs}(\tau_2) = \emptyset \quad \Gamma, \Phi, \mathcal{I} \vdash e_2 : \tau_1}{\Gamma, \Phi, \mathcal{I} \vdash e_1 \ e_2 : \tau_2} \text{TAPP1} \\
\\
\frac{\Gamma, \Phi, \mathcal{I} \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \text{refs}(\tau_2) \neq \emptyset \quad \Gamma, \Phi, \mathcal{I} \vdash e_2 : \text{Int}(\cdot) \quad \emptyset, \emptyset \vdash e_2 \rightsquigarrow \text{n} \quad \tau'_2 = \text{shift}(n, \tau_2)}{\Gamma, \Phi, \mathcal{I} \vdash e_1 \ e_2 : \tau'_2} \text{TAPP2}
\end{array}$$

Fig. 6. Typing rules for $\mathcal{L}_{DS}$ (expressions)

Rules for variables (TVAR), boolean constants (TBOOL), **let**-expressions (TLET) and pairs (TPAIR) are classical. The only difference is that the instantiation and generalisation functions (**Gen**, **Inst**) here operate both on type and size variables.

Rule TINT types integer constants, assigning a new, fresh size variable to the corresponding type.

Rule TCAST deals with type refinement. The refinability relation $<:$ is defined in Fig. 7. The first two cases deals with integers. The former correspond to a true “refinement”, when an hitherto unknown size is specified and the latter to resizing¹³. The last two cases extend the refinement relation to structured types.

$$\begin{array}{ccc}
\frac{}{\text{Int}(\beta) <: \text{Int}(\gamma)} & \frac{}{\text{Int}(\text{n}_1) <: \text{Int}(\text{n}_2)} & \frac{\tau_1 <: \tau'_1 \quad \tau_2 <: \tau'_2}{\tau_1 \times \tau_2 <: \tau'_1 \times \tau'_2} \qquad \frac{\tau'_1 <: \tau_1 \quad \tau_2 <: \tau'_2}{\tau_1 \rightarrow \tau_2 <: \tau'_1 \rightarrow \tau'_2}
\end{array}$$

Fig. 7. The type refinability relation for the $\mathcal{L}_{DS}$ language

¹³ Of course, resizing may induce a loss of precision at runtime if the new size is too small for the corresponding value.

Rule TFUN deals with the typing of functions. Arguments are scanned from left to right and appended to the index environment Υ (so that the i^{th} argument will appear at position i). Size variables occurring in type expressions are added to the variable environment Φ ¹⁴.

Rule TAPP1 describes “normal” applications, when the return type of applied function does not contain size references. This is detected by means of the auxilliary function `refs`. This function takes a type τ and returns the set of size references occurring in τ . It can be defined inductively as follows :

$$\begin{aligned} \text{refs}(\alpha) &= \emptyset & \text{refs}(\text{Int}\langle n \rangle) &= \emptyset & \text{refs}(\text{Int}\langle \beta \rangle) &= \emptyset & \text{refs}(\text{Int}\langle \bar{k} \rangle) &= \{k\} \\ \text{refs}(\tau_1 \times \tau_2) &= \text{refs}(\tau_1) \cup \text{refs}(\tau_2) & \text{refs}(\tau_1 \rightarrow \tau_2) &= \text{refs}(\tau_1) \cup \text{refs}(\tau_2) \end{aligned}$$

Rule TAPP2 describes the application of a function when the result type *does* contain size references. In this case, after checking that the type of argument is `Int`, this argument is *evaluated* and the return type of the application is updated accordingly. This updating is performed by the auxilliary function `shift`. This function takes an integer value n and a type τ and returns a type τ' where all occurrences of type `Int` $\langle \bar{1} \rangle$ have been substituted by `Int` $\langle n \rangle$ and all occurrences of type `Int` $\langle \bar{k} \rangle$, where $k > 1$, have been substituted by `Int` $\langle \overline{k-1} \rangle$. The `shift` function can be defined inductively as follows :

$$\begin{aligned} \text{shift}(n, \alpha) &= \alpha & \text{shift}(n, \text{Int}\langle n \rangle) &= \text{Int}\langle n \rangle & \text{shift}(n, \text{Int}\langle \beta \rangle) &= \text{Int}\langle \beta \rangle & \text{shift}(n, \text{Int}\langle \bar{1} \rangle) &= \text{Int}\langle n \rangle \\ \text{shift}(n, \text{Int}\langle \bar{k} \rangle) &= \text{Int}\langle \overline{k-1} \rangle \quad k > 1 & \text{shift}(n, \tau_1 \times \tau_2) &= \text{shift}(n, \tau_1) \times \text{shift}(n, \tau_2) \\ \text{shift}(n, \tau_1 \rightarrow \tau_2) &= \text{shift}(n, \tau_1) \rightarrow \text{shift}(n, \tau_2) \end{aligned}$$

The presence of the evaluation reduction rule $\rightsquigarrow$ in rule TAPP2 shows the intrication of static and dynamic semantics which is the hallmark of dependently typed languages¹⁵. But, compared to “full-fledged” dependently-typed languages, this intrication is here limited because the so-called *index language* concerns only the size annotations of one type (`int`) and cannot describe *computations*¹⁶.

2.2 Evaluation

Evaluation rules for $\mathcal{L}_{DS}$ are entirely standard except for those related to the type refinement operator. They are of the form $\rho, \Gamma \vdash e \rightsquigarrow v$ where ρ is an evaluation environment, mapping identifiers to semantic values and Γ the typing environment, mapping identifiers to types (the latter is required to handle type refinement, precisely). Semantic values are classical and include integer or boolean constants, pairs of values, closures, and primitives. The initial evaluation environment ρ_0 maps identifiers to the *builtin* primitives (`+`, `*`, `=`, `...`).

Fig. 8 depicts the two relevant rules.

Rule ECAST1 deals with the resizing of integer values, since the coercion may in this case have an effect on the corresponding value (for example, casting the value 15 to type `Int` $\langle 3 \rangle$ should give value 7 (assuming here, for the sake of simplicity, unsigned integers). This dynamic resizing operation is supposed to be carried out by the `resize` function (which ultimately depends on the implementation¹⁷). The corresponding rule also explains why the *typing* environment Γ belongs to the context of all evaluation rules.

¹⁴ It may seem that there’s some amount of redundancy here between Φ and Υ but having a separate variable index environment simplifies the writing of the rules dealing with type expressions.

¹⁵ Conversely, and as will be showed in Sec. 2.2, the dynamic evaluation rule for the type refinement operator uses the static typing rules.

¹⁶ It is important to note, in particular, that the evaluation step occurring in rule R4 happens in *empty* environments.

¹⁷ In CAPH, this function is provided by the VHDL functions `resize(unsigned,natural)` and `resize(signed,natural)` of the `numeric_std` IEEE library.

Rule ECAST2 deals with all other cases. The type parameter is there simply ignored, which is the usual strategy in most of ML-like languages.

$$\frac{\rho, \Gamma \vdash e \rightsquigarrow v \quad \Gamma, \emptyset, \emptyset \vdash t \Rightarrow \text{Int}\langle n \rangle, \Phi}{\rho, \Gamma \vdash e \triangleright t \rightsquigarrow \text{resize}(v, n)} \text{EC}_{\text{AST1}} \quad \frac{\rho, \Gamma \vdash e \rightsquigarrow v}{\rho, \Gamma \vdash e \triangleright t \rightsquigarrow v} \text{EC}_{\text{AST2}}$$

Fig. 8. The two relevant evaluation rules for $\mathcal{L}_{DS}$

2.3 Example

Fig 9 depicts the type derivation for the expressions

$$\text{fun } (n : \text{int}) \rightarrow \text{fun } (x : \text{int} \langle @n \rangle) \rightarrow x$$

and, then

f 8 0.

$$\begin{array}{c}
\frac{\frac{\frac{\Gamma_1(n) = \text{Int}\langle \cdot \rangle \quad \Upsilon_1(n) = 1}{\Gamma_1, \emptyset, \Upsilon_1 \vdash \text{int}\langle @n \rangle : \text{int}\langle \bar{1} \rangle, \emptyset} \text{R6} \quad \frac{\frac{\Gamma_2(x) = \forall. \text{Int}\langle \bar{1} \rangle}{\Gamma_1. \{x : \forall. \text{Int}\langle \bar{1} \rangle\}, \emptyset, \{n, x\} \vdash x : \text{Int}\langle \bar{1} \rangle} \text{TVAR}}{\Gamma_1. \{x : \forall. \text{Int}\langle \bar{1} \rangle\}, \emptyset, \{n, x\} \vdash x : \text{Int}\langle \bar{1} \rangle} \text{TFUN}} \\
\frac{\frac{\frac{\emptyset, \emptyset, \emptyset \vdash \text{int} : \text{Int}\langle \beta_1 \rangle, \emptyset}{\emptyset, \emptyset, \emptyset \vdash \text{fun} (n : \text{int}) \rightarrow \text{fun} (x : \text{int}\langle @n \rangle) \rightarrow x : \text{Int}\langle \beta_1 \rangle \rightarrow \text{Int}\langle \bar{1} \rangle \rightarrow \text{Int}\langle \bar{1} \rangle} \text{R5} \quad \frac{\frac{\frac{\Gamma_1}{\{n : \forall. \text{Int}\langle \beta_1 \rangle\}}, \emptyset, \{n\} \vdash \text{fun} (x : \text{Int}\langle @n \rangle) \rightarrow x : \text{Int}\langle \bar{1} \rangle \rightarrow \text{Int}\langle \bar{1} \rangle} \text{TFUN}}{\emptyset, \emptyset, \emptyset \vdash \text{fun} (n : \text{int}) \rightarrow \text{fun} (x : \text{int}\langle @n \rangle) \rightarrow x : \text{Int}\langle \beta_1 \rangle \rightarrow \text{Int}\langle \bar{1} \rangle \rightarrow \text{Int}\langle \bar{1} \rangle} \text{TFUN}} \\
\\
\frac{\frac{\frac{\Gamma, \emptyset, \emptyset \vdash f : \text{Int}\langle \beta \rangle \rightarrow \overbrace{\text{Int}\langle \bar{1} \rangle \rightarrow \text{Int}\langle \bar{1} \rangle}^{\tau_2}}{\text{refs}(\tau_2) = \{\bar{1}\}} \quad \frac{\Gamma, \emptyset, \emptyset \vdash 8 : \text{Int}\langle \cdot \rangle}{\emptyset, \emptyset \vdash 8 \rightsquigarrow 8}}{\text{shift}(8, \tau_2) = \text{Int}\langle 8 \rangle \rightarrow \text{Int}\langle 8 \rangle} \text{TAPP2} \quad \frac{\text{refs}(\text{Int}\langle 8 \rangle) = \emptyset \quad \frac{}{\Gamma, \emptyset, \emptyset \vdash 0 : \text{Int}\langle \beta_2 \rangle} \text{TINT}}{\frac{\Gamma}{\{f : \forall \beta. \text{Int}\langle \beta \rangle \rightarrow \text{Int}\langle \bar{1} \rangle \rightarrow \text{Int}\langle \bar{1} \rangle\}, \emptyset, \emptyset \vdash f \ 8 \ 0 : \text{Int}\langle 8 \rangle} \text{TAPP1}}
\end{array}$$

Fig. 9. Example of type derivation in $\mathcal{L}_{DS}$

2.4 Application

We now give practical examples of how the type system described above is actually used in CAPH programs.

The first example involves the **resize** introduced in Sec. 1. Fig. 10 shows a small program and the corresponding dataflow graph representation computed by the CAPH compiler. The **resize** actor is defined in lines 1–6. The **k** parameter is used to give the size (in bits) of its output **o**. The size of its input, **n**, is left unspecified (*i.e.* generic). In this example, integer size is specified as the second parameter of the **int** type constructor. The first parameter gives its *signness*. The given signature therefore allows the **resize** actor to be applied both to **signed** and **unsigned** integers but ensures that input and output have the same signness. The behavior the **resize** actor is specified lines 4–7 with a single transition rule. This rule simply applies the type refinement operator “:” to the value read on the input to produce the value written on the output. The **resize** actor is instantiated twice in lines 13–14, with a different input type (resp. **signed**<8> and **unsigned**<10>) and parameter value (resp. 4 and 12).

```

1 actor resize (k:int)
2   in (a:int<s,n>)
3   out (c:int<s,k>)
4 rules
5 | a:x -> c:(x:int<s,k>)
6 ;
7
8 stream i1:signed<8> from "inp1.txt";
9 stream o1:signed<4> to "res1.txt";
10 stream i2:unsigned<10> from "inp2.txt";
11 stream o2:unsigned<12> to "res2.txt";
12
13 net o1 = resize 4 i1;
14 net o2 = resize 12 i2;
```

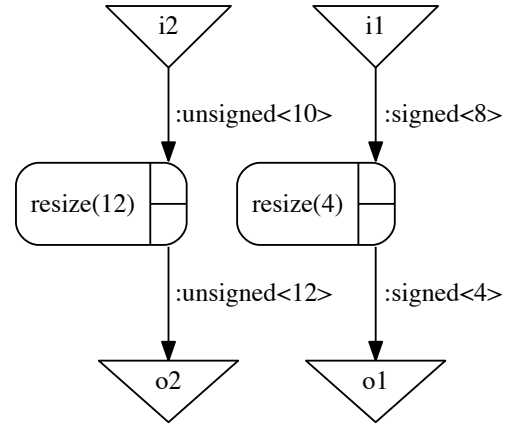


Fig. 10. A CAPH program involving the **resize** actor

A second, more complex, example is given in Fig. 11. This example involves the **dk** actor. As specified in lines 1–3, this actor has two parameters, **k** and **v**, one input and one output. The **k** parameter, the input and the have the same (generic) type **\$t**. The role of the **dk** actor is to delay its input stream by inserting **k** predefined tokens having value **v** at the beginning. For instance, if the input stream is

1 2 3 4 5 6 7 8 ...

then the output stream, say, for $k = 4$ and $v = 0$, will be

0 0 0 0 1 2 3 4 5 6 7 8 ...

We will not describe in detail the behavior of the actor, as specified in lines 8–10 (this behavior is basically specified as two-states FSM¹⁸). For what is concerned here, it suffices to note that it relies on two local

¹⁸ The interested reader is referred to [1].

variables, an array **z** and an index **i**. The array is, classically, used as a circular buffer, memorizing the k^{th} previous values of the input stream. The key characteristic here is that the *size* of the **z** array, consequently, depends on the value of the **k** parameter. This is precisely what is reflected by the declaration at line 5. Prior to the integration of dependent type system described here to the CAPH language, and because there's no such thing as dynamically allocated arrays in VHDL, the programmer had to decide *a priori* of a size for the **z** array when writing actors such as **dk**. This could result either to a waste of resources (if this size was over-estimated), or, more annoyingly, to runtime errors (if this size was under-estimated). This situation occurred quite frequently in the process of writing the CAPH standard libraries of actors and was indeed a strong motivation for integrating the extensions described in this paper to the language.

```

1  actor dk (k:int , v:$t)
2    in (a:$t )
3    out (c:$t)
4  var s : {S0,S1} = S0
5  var z : $t array[k] = [ v | i = 0 to k-1 ]
6  var i : int = 0
7  rules
8  | (s:S0, a:p) when i<k-1 -> (s:S0, c:v, i:i+1, z[i]:p)
9  | (s:S0, a:p)                -> (s:S1, c:v, i:0, z[i]:p)
10 | (s:S1, a:p)                -> (s:S1, c:z[i], i:(i+1) mod k, z[i]:p)
11 ;
12
13 stream i:unsigned<8> from "sample.txt";
14 stream o:unsigned<8> to "result.txt";
15
16 net o = dk (4,0) i;

```

Fig. 11. A CAPH program involving the **dk** actor

3 Related work

There has been a considerable amount of research on the topic of dependent types. As pointed out by Norell in [4], the related work can be roughly divided in two families : the approaches starting from type theory and trying to turn it into programming languages and those starting from an existing programming language and trying to add some desired type theoretic features to it. Our work clearly belongs to the second category. It is more precisely related to the notion of *indexed types*, as studied for example by Hugues, Pareto and Sabry in [5], Xi and Pfenning in [6], Zenger [7] or Chin and Khoo [8]. Basically, some type constructors are indexed with objects drawn from an integer constraint domain with linear equalities, inequalities and a subset of arithmetic relations and type checking/inference can be carried out by solving a constraint satisfaction problem (CSP). The constraint domain and associated relations are chosen so that interesting properties can be expressed and checked while keeping the CSP practically solvable. For example, by restricting arithmetic relations between indices to Presburger formulae, Xi and Pfenning [6] showed how some program verification and optimisation steps such as array bound check elimination or dead code elimination can be performed at compile time. Similarly, Zenger [7] showed how the technique can be applied to statically detect incompatibility between vector or matrix sizes in programs. The work of Hugues and Pareto [5] used sized types to check to correctness of reactive programs manipulating recursively defined data structures. In [9], a similar approach is

followed for automatically inferring upper bounds for sized types in the context of static evaluation of program costs.

Our goals, comparatively, are much more modest and, in a sense, more pragmatic. In particular, we deliberately restricted the index domain to integers and relations between indices to equalities, so that type inference can be carried out by simple unification (extended here to size variables), hence avoiding the need for a constraint solver. The mechanism of size reference, as introduced here can also be viewed as a way of restricting the interactions between the aspects related to simple parametric polymorphism and those intrinsically related to dependent typing (as already noted by Zenger in [7], complexity essentially stems from the combination of indexed and dependent types in the same framework). The nature of the target language for CAPH (VHDL) of course strongly influenced these design decisions. Incidentally, size-generic functions are indeed a common notion in VHDL. For example, the "+" operator is defined in the IEEE `numeric_std` library as `function "+" (a, b: unsigned) return unsigned`, where the size (in bits) of the arguments and result is an implicit generic parameter (meaning that this operator can be used to add unsigned integers of any size). The notion of size variable presented here can be viewed as a way of “lifting” this feature up to our source language without compromising the elegance and safety of Hindley-Milner type inference. In the same vein, dependent types in CAPH essentially serve the need to statically parameterize actors at the network level, meaning that the types depending on the parameter values can only be attached to inputs, outputs or local variables. This justifies the proposed mechanism of size references.

Our work can also be related to those on so-called *number parameterized types*, as described for instance in [10], where the author describes the realisation of a system supporting types indexed by integer values based upon a phantom type representation of indices expressed as a sequence of decimal digits. The use of clever encodings clever encoding allows expression of equality and also simple inequality and arithmetic constraints on indices. However the presented ideas were implemented as an Haskell library which prevented their inclusion in our pre-existing DSL.

4 Conclusion

We have presented a lightweight type system supporting a limited form of dependently sized types. This type system, first prototyped in Caml is now integrated to the CAPH programming language, the concerned types being `int`, `signed`, `unsigned` and `array`. Though simple, it had a significant impact on the expressive power of the language, allowing, in particular a more abstract version of the standard libraries (dedicated to signal and image processing) than in the previous version to be written. Both the Caml prototype and the full CAPH language are freely available on the Web (at caph.univ-bpclermont.fr/extra/depdttypes.tar.gz and caph.univ-bpclermont.fr resp.).

We are now contemplating the idea of extending the language to accomodate simple arithmetic relations between type sizes. This could be useful for modeling certain operations like size-extending addition (`add: int<s> * int<s> -> int<s+1>`) or multiplication (`add: int<s> * int<s> -> int<s*2>`). It is not clear at this point, however, whether this can be done without going into the complexity of CSP-based dependently typed languages.

References

1. J. Sérot. The caph software and reference manual. <http://caph.univ-bpclermont.fr>.
2. J. Sérot, F. Berry, C. Bourrasset. High-level dataflow programming for real-time image processing on smart cameras. *Journal of Real-Time Image Processing*, 2014.
3. J. Sérot, F. Berry, and S. Ahmed. Implementing stream-processing applications on fpgas: A dsl-based approach. In *Proceedings of the 2011 21st International Conference on Field Programmable Logic and Applications, FPL '11*, pp 130–137.

4. U. Norell. Towards a practical programming language based on dependent type theory. PhD thesis, Department of Computer Science and Engineering, Chalmers University of Technology, 2007.
5. J. Hughes, L. Pareto, A. Sabry. Proving the Correctness of Reactive Systems Using Sized Types. Proceedings of the 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, 1996, pp 410-423.
6. H. Xi, F. Pfenning. Dependent Types in Practical Programming. Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, 1999, pp 214-227.
7. C. Zenger. Indexed Types. J. Theor. Comput. Sci., 187(1), 1997, pp 147-165.
8. W. Chin, S. Khoo. Calculating Sized Types. SIGPLAN Not., 34(11), 1999, pp 62-72.
9. A. Portillo, K. Hammond, H.W. Loidl, P. Vasconcelos. Cost Analysis Using Automatic Size and Time Inference. Implementation of Functional Languages, LNCS 2670, 2003, pp 232-247.
10. O. Kiselyov. Number-parameterized types.